

Refactoring Plan: Project Library

Table of Contents

1. Overview	2
1.1 Purpose	2
1.2 Scope	2
1.3 Prerequisites	3
1.4 Stages	3
1.5 Version history	3
2. Change plan	3
2.1 Summary	3
2.2 Change 1: License headers and javadoc	4
2.3 Change 2: Typed dates (fixes Error #10)	4
2.4 Change 3: Loan policy (fixes Errors #5 and #11)	5
2.5 Change 4: Not-found error (fixes Errors #9 and #12)	5
2.6 Change 5: Base DAO (supports Errors #3 and #8)	5
2.7 Change 6: Book service guard (fixes Error #6)	6
2.8 Change 7: Member service cleanup (fixes Error #2)	7
2.9 Change 8: Loan service (fixes Errors #2, #3, #4, #12)	7
2.10 Change 9: Reservation service (fixes Errors #7 and #8)	8
2.11 Change 10: Database seed	8
2.12 Change 11: Application wiring	9
2.13 Change 12: Controllers	9
2.14 Change 13: Browser overdue state	10
2.15 Change 14: Logback level	10
2.16 Change 15: Tests	11
2.17 Change 16: Class diagram	11
3. Verification	12
3.1 Build and run	12
3.2 Test matrix	12
3.3 Definition of done	12
4. Risks	12
5. Architecture error analysis	13
5.1 Summary	13
5.2 Error #1: Layering violation (controller bypasses the service layer)	14
5.3 Error #2: God service	14
5.4 Error #3: Missing transaction	14
5.5 Error #4: Cross-module coupling (no single owner of <code>availableCopies</code>)	15
5.6 Error #5: Duplicated business rule (loan period)	15

5.7 Error #6: Checkout has no availability guard (copies can go negative)	15
5.8 Error #7: Fulfill never updates the inventory (copy count drift)	16
5.9 Error #8: Missing transaction in fulfill (third site)	16
5.10 Error #9: No null checks at the trust boundary	16
5.11 Error #10: Dates stored as String and wall clock in the domain	17
5.12 Error #11: Hardcoded overdue fee rate	17
5.13 Error #12: returnLoan hides a missing or returned loan	17
6. Grading rubric	18
6.1 Part 1: Data layer and policy (Stage 1, 38 points)	18
6.2 Part 2: Services and integration (Stage 2, 42 points)	19
Appendix A: Prompt log format	20

Field	Value
Document	Refactoring plan
Version	1.1
Status	Approved for implementation
Date	2026-09-22
Owner	Arquitectura de Sistemas, DISC, UCN, Antofagasta
Target	<code>src/</code>
Related documents	<code>README.md</code> , <code>docs/class-diagram.puml</code>
Total points	80

1. Overview

1.1 Purpose

This document defines the refactoring plan for Project Library. The plan removes 12 architecture errors from the current code.

- Section 2 gives the ordered changes.
- Section 5 explains each error.
- Section 6 gives the grading rubric.

1.2 Scope

In scope

- The 16 changes in Section 2.
- The model, the persistence layer, the policy, the services, the controllers, the application wiring, the browser overdue state, the logging level, and the tests.

Out of scope

- New features.
- New dependencies. Keep the current `build.gradle`.

1.3 Prerequisites

- Use Java 21 or later. `Database.seedIfEmpty()` uses `List.getFirst()`.
- Keep the current `build.gradle`. It already includes `jackson-datatype-jsr310` and `logback-classic`.

1.4 Stages

The work has two stages. One student does Stage 1 first, then Stage 2. Part 1 grades Stage 1. Part 2 grades Stage 2.

- **Stage 1 (data layer and policy):** Changes the model, the persistence layer, and the shared infrastructure. The services keep their current shape, but they must compile against the new types. It also points `MemberService.checkout()` and `ReservationService.fulfill()` at `LoanPolicy.dueDate()`.
- **Stage 2 (services and integration):** Moves the lending logic into the services, adds the clock and the transactions, rewires the application, and updates the tests.

Each stage ends with a build and a test run. Both stages end at 23:59 America/Santiago. Section 6 gives the points per stage.

1.5 Version history

Version	Date	Change
1.0	2026-09-22	First issue.
1.1	2026-09-22	Add the stage dates. Resolve Error #12.

2. Change plan

2.1 Summary

The table below lists the 16 changes. The Stage column gives the stage that applies the change. Each number has a section below.

Number	Stage	Area	Change
1	1, 2	all files	Add license header and javadoc
2	1	dates	Store dates as <code>LocalDate</code>
3	1	policy	Add <code>LoanPolicy</code>

Number	Stage	Area	Change
4	1	errors	Add <code>NotFoundException</code>
5	1	DAO	Add <code>BaseDao</code> , simplify the four DAOs
6	1	<code>BookService</code>	Guard the inventory
7	2	<code>MemberService</code>	Remove <code>checkout()</code>
8	2	<code>LoanService</code>	Own <code>checkout()</code> , use a clock and transactions
9	2	<code>ReservationService</code>	Use transactions and <code>BookService.borrow()</code>
10	1	<code>Database</code>	Change the seed data
11	2	<code>App</code>	Change the wiring and the server setup
12	2	controllers	Route the calls through the services
13	1	<code>app.js</code>	Compute the overdue state in the browser
14	1	<code>logback.xml</code>	Set the debug level
15	2	tests	Update two tests, add two tests
16	1, 2	diagram	Update <code>docs/class-diagram.puml</code>

2.2 Change 1: License headers and javadoc

Files: every Java file, `index.html`, `logback.xml`.

1. Add this header to every Java file:

```
/*
 * Copyright (c) 2026. Arquitectura de Sistemas, DISC, UCN, Antofagasta.
 */
```

2. Add the same header to `index.html` and `logback.xml` as an XML comment. `app.js` has no header.
3. Add javadoc to every type, field, method, and parameter. Describe each parameter with `@param`, each return value with `@return`, and each exception with `@throws`.

2.3 Change 2: Typed dates (fixes Error #10)

Files: `model/Loan.java`, `model/Reservation.java`, new `db/LocalDatePersister.java`.

1. Change `Loan.loanDate`, `Loan.dueDate`, `Loan.returnDate`, and `Reservation.reservedAt` from `String` to `LocalDate`. ORMLite persists a `LocalDate` with a custom persister.
2. Create `db/LocalDatePersister.java`. Extend `BaseDataType`. Use the SQL type `STRING` and register `LocalDate.class`. Add the four overrides:
 - `parseDefaultString` returns the input string.
 - `resultToSqlArg` returns `results.getString(columnPos)`.

- `sqlArgToJava` returns `LocalDate.parse((String) sqlArg)`.
- `javaToSqlArg` returns `javaObject.toString()`.

Use a private constructor and a static singleton.

3. In `model/Loan.java` and `model/Reservation.java`, add the persister to each date field:

```
@DatabaseField(canBeNull = false, persisterClass = LocalDatePersister.class)
private LocalDate dueDate;
```

4. Change the constructors, the getters, and the setters to `LocalDate`.
5. Delete `Loan.isOverdue()`. The service and the browser compute the overdue state.

2.4 Change 3: Loan policy (fixes Errors #5 and #11)

Files: new `service/LoanPolicy.java`.

Move the loan period and the fee into one class. This class removes the disagreement between 14 and 21 days.

```
public static final int DUE_DAYS = 21;
public static final double FEE_PER_DAY = 1.0;

public static LocalDate dueDate(LocalDate loanDate) {
    return loanDate.plusDays(DUE_DAYS);
}
```

Add a private constructor.

Replace `plusDays(14)` in `MemberService.checkout(...)` and `plusDays(21)` in `ReservationService.fulfill(...)` with `LoanPolicy.dueDate(...)`. `DueDateDuplicationTest` passes at the end of Stage 1. Stage 2 moves the two call sites into the services.

2.5 Change 4: Not-found error (fixes Errors #9 and #12)

Files: new `service/NotFoundException.java`.

Extend `RuntimeException`. Add one constructor with a `String message` parameter.

2.6 Change 5: Base DAO (supports Errors #3 and #8)

Files: new `dao/BaseDao.java`, `dao/BookDao.java`, `dao/MemberDao.java`, `dao/LoanDao.java`, `dao/ReservationDao.java`.

`BaseDao<T>` holds the shared CRUD code. The four DAOs extend it.

1. Add a protected `Dao<T, Integer> dao` field. The constructor calls `DaoManager.createDao(connectionSource, clazz)`.
2. The methods `findAll`, `findById`, `create`, `update`, and `delete` catch `SQLException` and throw `RuntimeException`.
3. The `transaction` method is the important one:

```
public <R> R transaction(Callable<R> callable) throws SQLException {
    try {
        return TransactionManager.callInTransaction(dao.getConnectionSource(),
            callable);
    } catch (SQLException e) {
        if (e.getCause() instanceof RuntimeException cause) {
            throw cause;
        }
        throw e;
    }
}
```

The `unwrap` keeps the domain error (for example `NotFoundException`) after a failed transaction.

Each DAO becomes one small class. `BookDao` is the example:

```
public final class BookDao extends BaseDao<Book> {
    public BookDao(ConnectionSource connectionSource) {
        super(connectionSource, Book.class);
    }
}
```

`MemberDao`, `LoanDao`, and `ReservationDao` are the same with their own entity class. The constructors and the methods no longer throw `SQLException`.

2.7 Change 6: Book service guard (fixes Error #6)

Files: `service/BookService.java`.

1. Remove `throws SQLException` from every method.
2. `borrow()` rejects a missing book and a book with no copies:

```
Book book = dao.findById(bookId);
if (book == null) {
    throw new NotFoundException("Book not found: " + bookId);
}
if (book.getAvailableCopies() <= 0) {
    throw new IllegalStateException("No available copies of book " + bookId);
}
```

3. `returnCopy()` rejects a missing book.
4. Both methods change the copy count and call `dao.update(book)`.

2.8 Change 7: Member service cleanup (fixes Error #2)

Files: `service/MemberService.java`.

1. Delete `checkout()` and the `BookDao` and `LoanDao` fields. Keep only the `MemberDao`:

```
public MemberService(MemberDao memberDao) {  
    this.memberDao = memberDao;  
}
```

2. Remove `throws SQLException` from `register()` and `findAll()`.

2.9 Change 8: Loan service (fixes Errors #2, #3, #4, #12)

Files: `service/LoanService.java`.

The service owns `checkout()` now. It receives the `MemberDao`, the `BookService`, and a `Clock`:

```
public LoanService(LoanDao loanDao, MemberDao memberDao, BookService bookService,  
    Clock clock) {
```

Apply these changes:

- `findAll()` returns `withAccruedFees(loanDao.findAll())`.
- `returnLoan()` throws `NotFoundException` for a missing loan and `IllegalStateException` for a returned loan.
- `returnLoan()` runs the loan update and `bookService.returnCopy(...)` in one transaction.
- `returnLoan()` sets the fee only when `today.isAfter(loan.getDueDate())`.
- `overdueLoans()` filters `!loan.isReturned()` and `loan.getDueDate().isBefore(today)`.
- `withAccruedFees()` sets the fee on every open overdue loan. It does not persist the fee.
- `accruedFee()` returns `ChronoUnit.DAYS.between(loan.getDueDate(), today) * LoanPolicy.FEE_PER_DAY`.
- `checkout()` checks the member and the book, then runs `bookService.borrow(bookId)` and `loanDao.create(loan)` in one transaction.
- `checkout()` creates the loan with `LoanPolicy.dueDate(today)`.
- Delete the old `DUE_DAYS` constant. `LoanPolicy` holds it.

Read every date with `LocalDate.now(clock)`.

2.10 Change 9: Reservation service (fixes Errors #7 and #8)

Files: `service/ReservationService.java`.

The service receives the `MemberDao`, the `LoanDao`, the `BookService`, and a `Clock`:

```
public ReservationService(ReservationDao reservationDao, MemberDao memberDao, LoanDao
loanDao,
                        BookService bookService, Clock clock) {
```

Apply these changes:

- `reserve()` rejects a missing book and a missing member with `NotFoundException`.
- `reserve()` uses `new Reservation(member, book, LocalDate.now(clock))`.
- `fulfill()` rejects a missing reservation with `NotFoundException` and a fulfilled reservation with `IllegalStateException`.
- `fulfill()` runs the reservation update, `bookService.borrow(...)`, and the loan creation in one transaction.
- `fulfill()` uses `LoanPolicy.dueDate(today)`.
- Delete the `BookDao` field.

2.11 Change 10: Database seed

Files: `db/Database.java`.

Keep the table creation. Change the seed:

- Create three members, not six.
- Create one reservation, not two.
- Create three loans: one returned, one active, and one overdue.
- Decrease `availableCopies` only for the active loan and the overdue loan. A returned loan does not change the count.
- Log each step with `log.debug(...)`.
- Delete the `createLoan` helper method.

The returned loan uses `setReturned(true)` and `setReturnDate(...)`:

```
Loan returned = new Loan(members.getFirst(), books.getFirst(), today.minusDays(30),
today.minusDays(9));
returned.setReturned(true);
returned.setReturnDate(today.minusDays(10));
```

```
loanDao.create(returned);
```

Add a logger field:

```
private static final Logger log = LoggerFactory.getLogger(Database.class);
```

2.12 Change 11: Application wiring

Files: `App.java`.

Change the wiring:

- Use `jdbc:sqlite:database.db`, not `database.sqlite`.
- Create the clock with `Clock.system(ZoneId.of("America/Santiago"))`.
- `MemberService` receives only the `MemberDao`.
- `LoanService` and `ReservationService` receive the `BookService` and the clock.
- `BookController` receives only the `BookService`. `LoanController` receives only the `LoanService`.
- Log each step with `log.info(...)`. Add a logger field.

Configure the JSON mapper to write `LocalDate` values as strings:

```
config.jsonMapper(new JavalinJackson().updateMapper(mapper ->
    mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)));
```

Add the exception handlers:

```
config.routes.exception(NotFoundException.class, (e, ctx) ->
    ctx.status(404).json(Map.of("error", e.getMessage())));
config.routes.exception(IllegalArgumentException.class, (e, ctx) ->
    ctx.status(400).json(Map.of("error", e.getMessage())));
config.routes.exception(IllegalStateException.class, (e, ctx) ->
    ctx.status(409).json(Map.of("error", e.getMessage())));
```

Keep the static file configuration. Start the server with `app.start(7070)`.

2.13 Change 12: Controllers

Files: `controller/BookController.java`, `controller/LoanController.java`,
`controller/ReservationController.java`, `controller/MemberController.java`.

Book controller (fixes Error #1)

Delete the `BookDao` field and the constructor parameter. The `GET /books` route calls the service:

```
config.routes.get("/books", ctx -> ctx.json(service.listAll()));
```

Loan controller (fixes Error #2)

Delete the `MemberService` field and the constructor parameter. The `POST /loans` route calls the loan service:

```
ctx.json(loanService.checkout(memberId, bookId));
```

Reservation controller

Delete the two `Objects.requireNonNull` calls and the `java.util.Objects` import:

```
int memberId = Integer.parseInt(ctx.queryParam("memberId"));
int bookId = Integer.parseInt(ctx.queryParam("bookId"));
```

Member controller

No functional change. Add the license header and the javadoc.

2.14 Change 13: Browser overdue state

Files: `resources/public/app.js`, `resources/public/index.html`.

The `Loan` JSON no longer has the `overdue` property. Compute it in the browser. Add these lines after the `esc` helper:

```
const now = new Date();
const today = `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, '0')}-${String(now.getDate()).padStart(2, '0')}`;
const isOverdue = (l) => !l.returned && l.dueDate < today;
```

Replace the two uses of `l.overdue` with `isOverdue(l)`.

`index.html` receives only the license comment.

2.15 Change 14: Logback level

Files: `resources/logback.xml`.

Add the license comment and this logger:

```
<logger name="cl.ucn.disc" level="DEBUG"/>
```

2.16 Change 15: Tests

Files: `DueDateDuplicationTest.java`, `TransactionBugTest.java`, new `AccruedFeeTest.java`, new `DatabaseSeedTest.java`.

`DueDateDuplicationTest.java`

Replace `MemberService` with `LoanService`. Use a fixed clock for both services:

```
Clock clock = Clock.fixed(Instant.parse("2026-01-15T10:00:00Z"), ZoneOffset.UTC);
```

Create the test book with two copies. `checkout()` and `fulfill()` each borrow one copy. Call `loanService.checkout(...)` in the test method.

`TransactionBugTest.java`

Replace `MemberService` with `LoanService`. Delete the unused `@Disabled` import. Expect `NotFoundException` from the failed checkout:

```
assertThrows(NotFoundException.class, () -> loanService.checkout(9999, book.getId()));
```

New file `AccruedFeeTest.java`

Create an in-memory database, a book, and a member. Use a fixed clock. Create one overdue loan (due 2025-12-22) and one active loan. Call `loanService.findAll()`. Assert 24.0 for the overdue loan and 0.0 for the active loan.

New file `DatabaseSeedTest.java`

Create an in-memory database and call `db.seedIfEmpty()`. Assert that loans and reservations exist, that one loan is overdue, and that every book with an open loan has fewer available copies. Call `db.seedIfEmpty()` again. Assert that the number of loans and reservations does not change.

2.17 Change 16: Class diagram

Files: `docs/class-diagram.puml`.

Update the diagram after each stage:

- Stage 1: add `BaseDao`, `LocalDatePersister`, `LoanPolicy`, and `NotFoundException`. Show that the four DAOs extend `BaseDao`. Change the date fields to `LocalDate`. Delete `Loan.isOverdue()`.
- Stage 2: update `MemberService`, `LoanService`, and `ReservationService` with the new fields and methods. Update the controllers.

3. Verification

3.1 Build and run

```
./gradlew build
./gradlew run
```

Open <http://localhost:7070>.

3.2 Test matrix

Run `./gradlew build` after each stage. The table below maps each test to the errors it detects.

Test	Detects	Passes at
<code>DueDateDuplicationTest</code>	Error #5	End of Stage 1
<code>TransactionBugTest</code>	Errors #3 and #9	End of Stage 2
<code>AccruedFeeTest</code>	Error #11	End of Stage 2
<code>DatabaseSeedTest</code>	Errors #7 and #8	End of Stage 2

`DueDateDuplicationTest` must pass at the end of Stage 1. `TransactionBugTest` can still fail after Stage 1: Stage 2 fixes the transactions. All four tests must pass at the end of Stage 2. Inspect the code for the other errors.

3.3 Definition of done

The refactor is complete when:

- ☐ Every change in Section 2 is applied.
- ☐ Every error in Section 5 is closed.
- ☐ `./gradlew build` passes.
- ☐ All four tests pass.
- ☐ `docs/prompts-stage-1.md` and `docs/prompts-stage-2.md` exist and match the code.

4. Risks

The table below lists the main risks of the plan.

Risk	Impact	Mitigation
The <code>LocalDate</code> change breaks the call sites	Stage 1 does not compile	Stage 1 updates <code>MemberService</code> , <code>LoanService</code> , <code>ReservationService</code> , and <code>Database</code> . Run the build.

Risk	Impact	Mitigation
The transaction unwrap hides a domain error	A client receives a 500 response, not a 404	<code>BaseDao.transaction</code> rethrows the <code>RuntimeException</code> cause. <code>TransactionBugTest</code> detects this defect.
The JSON date format changes	The browser cannot compare the dates	App disables <code>WRITE_DATES_AS_TIMESTAMPS</code> . <code>app.js</code> compares ISO date strings.
The new seed changes the record counts	A manual check that assumes the old counts fails	Change 10 gives the exact counts. Change 15 adds <code>DatabaseSeedTest</code> to pin them.

5. Architecture error analysis

5.1 Summary

The original code has 12 architecture errors. Each error gives its location, why it is wrong, its impact, and how to fix it. The "Fixed by" column refers to the change sections in Section 2. The "Stage" column gives the stage that removes the error. A value of "1, 2" means that Stage 1 adds the mechanism and Stage 2 completes the fix.

#	Error	Category	Impact	Fixed by	Stage
1	Layering violation	Layering	<code>GET /books</code> bypasses the service layer	12	2
2	God service	Cohesion	One class owns the member lifecycle and lending	7, 8, 12	2
3	Missing transaction	Data integrity	A failed write leaves a partial copy decrement	5, 8	1, 2
4	Cross-module coupling	Coupling	Two modules write <code>availableCopies</code> behind the catalog	6, 8, 9	1, 2
5	Duplicated loan period	Duplication	Checkout and fulfill disagree: 14 against 21 days	3, 8, 9	1
6	No availability guard	Domain invariant	<code>availableCopies</code> can go negative	6, 8	1, 2
7	Fulfill skips the inventory	Domain invariant	A fulfilled reservation keeps the copy available	9	2
8	Missing transaction in fulfill	Data integrity	A fulfilled reservation can stay without a loan	5, 9	1, 2
9	No trust-boundary checks	Validation	A bad ID gives a 500 and a partial write	4, 6, 8, 9, 11	2
10	String dates and wall clock	Domain model	Date logic is text-based and untestable	2, 8, 9, 11	1, 2

#	Error	Category	Impact	Fixed by	Stage
11	Hardcoded fee rate	Policy	The rate is a magic number; open fees show 0.00	3, 8	2
12	Silent return	API contract	A missing or repeated return looks like success	4, 8, 11	2

5.2 Error #1: Layering violation (controller bypasses the service layer)

- **Location:** `controller/BookController.java`: the `GET /books` handler calls `dao.findAll()` instead of `service.listAll()`.
- **Why it is wrong:** the HTTP layer reaches into persistence. The controller depends on the DAO, and no single place enforces catalog rules. `BookService.listAll()` is dead code.
- **Impact:** the `GET /books` response comes straight from the DAO. A catalog rule (for example "show only available books") has no place to live, and the service layer is optional for this route.
- **Fix:** route the call through `BookService.listAll()`.
- **Test:** none.

5.3 Error #2: God service

- **Location:** `service/MemberService.java`: the class registers members and also runs `checkout(...)`. `controller/LoanController.java` sends `POST /loans` to `MemberService`.
- **Why it is wrong:** one class carries two unrelated jobs: the member lifecycle and lending. A change to the checkout rules forces a change to the member service.
- **Impact:** `MemberService` holds three DAOs and two jobs. Each new lending rule makes the class larger, and a checkout change touches the member module.
- **Fix:** move `checkout(...)` into `LoanService`.
- **Test:** none. `TransactionBugTest` calls the misplaced `MemberService.checkout(...)`.

5.4 Error #3: Missing transaction

- **Location:** `MemberService.checkout(...)` and `LoanService.returnLoan(...)`. Both methods do two writes (adjust `books.availableCopies` and insert or update a `loans` row) with no transaction.
- **Why it is wrong:** if the second write fails, the first write stays in the database. The copy count then drifts from the loan table. A checkout with a bad member ID shows this defect.
- **Impact:** a failed checkout with a bad member ID decrements `availableCopies` and creates no loan. The catalog then reports one copy fewer than reality.
- **Fix:** run both writes in `TransactionManager.callInTransaction(connectionSource, callable)`.
- **Test:** `TransactionBugTest` fails on this defect.

5.5 Error #4: Cross-module coupling (no single owner of `availableCopies`)

- **Location:** `MemberService.checkout(...)` and `LoanService.returnLoan(...)` change `Book.availableCopies` through `BookDao`. `BookService.borrow(...)` and `BookService.returnCopy(...)` are dead code.
- **Why it is wrong:** the catalog module owns the copy count, but two other modules write it behind the module interface. No single point enforces the invariant.
- **Impact:** the catalog cannot guard its own counter. `availableCopies` can go negative (Error #6) or drift from the loans (Error #7), because two services write the field directly.
- **Fix:** let `LoanService` call `BookService.borrow()` and `BookService.returnCopy()`. Delete the direct `BookDao` writes.
- **Test:** none.

5.6 Error #5: Duplicated business rule (loan period)

- **Location:** `MemberService.checkout(...)` uses `plusDays(14)`. `ReservationService.fulfill(...)` uses `plusDays(21)`. `LoanService.DUE_DAYS = 21` is declared but never read.
- **Why it is wrong:** checkout and fulfill create the same kind of loan with different due dates (14 days against 21 days). The unused constant cannot keep the two sites together. `DueDateDuplicationTest` fails on this difference.
- **Impact:** a member who checks out gets 14 days, and a member who reserves and fulfills gets 21 days for the same loan. A policy change needs an edit at each site.
- **Fix:** put one policy in `LoanPolicy` (`DUE_DAYS`, `dueDate(...)`) and use it at both sites. Delete `LoanService.DUE_DAYS`.
- **Test:** `DueDateDuplicationTest` fails on this difference.

5.7 Error #6: Checkout has no availability guard (copies can go negative)

- **Location:** `service/MemberService.java: checkout(...)` decreases `Book.availableCopies` with no check for zero.
- **Why it is wrong:** a book with no available copies can still go out, and the counter goes negative.
- **Impact:** the catalog reports a negative stock, and the library lends more copies than it owns.
- **Fix:** reject the checkout when `book.getAvailableCopies() <= 0`. Put the guard in `BookService.borrow()` so every inventory change passes through one method.
- **Test:** none.

5.8 Error #7: Fulfill never updates the inventory (copy count drift)

- **Location:** `service/ReservationService.java: fulfill(...)` creates a `Loan` and never decreases `Book.availableCopies`.
- **Why it is wrong:** checkout and fulfill both create a loan, but only checkout changes the copy count. After a fulfill, a loan exists with no matching decrement, so the inventory drifts.
- **Impact:** after a fulfill, the catalog still offers a copy that is already on loan. The same copy can go out twice.
- **Fix:** route `fulfill(...)` through `BookService.borrow()` so checkout and fulfill share one inventory path.
- **Test:** none.

5.9 Error #8: Missing transaction in fulfill (third site)

- **Location:** `service/ReservationService.java: fulfill(...)` does two writes (`reservationDao.update` and `loanDao.create`) with no transaction.
- **Why it is wrong:** the same defect as Error #3 at a third site. A failure between the writes leaves a fulfilled reservation with no loan, or the opposite.
- **Impact:** a failed fulfill marks the reservation as fulfilled and creates no loan. The member loses the reservation, and the library records no loan.
- **Fix:** run both writes in `TransactionManager.callInTransaction(connectionSource, callable)`.
- **Test:** none.

5.10 Error #9: No null checks at the trust boundary

- **Location:** `MemberService.checkout(...)` and `ReservationService.reserve(...)` read a member and a book, then write with no null check. `ReservationService.fulfill(...)` checks the reservation, but throws `IllegalStateException` when the record is missing.
- **Why it is wrong:** ORMLite `queryForId` returns null for a missing ID. A bad ID gives an unhandled exception and a 500 response. In checkout, the copy decrement happens before the failure, so the database keeps a partial write. In fulfill, the client cannot separate a missing reservation (404) from a fulfilled one (409).
- **Impact:** a bad member or book ID gives a 500 response, not a 400 or 404. In checkout, the copy decrement already happened, so the request also corrupts the inventory.
- **Fix:** check each record for null before the first write. Throw `NotFoundException` for a missing record and map it to 404. Map `IllegalArgumentException` to 400.
- **Test:** `TransactionBugTest` reaches this path with member ID 9999.

5.11 Error #10: Dates stored as String and wall clock in the domain

- **Location:** `model/Loan.java` (`loanDate`, `dueDate`, `returnDate`) and `model/Reservation.java` (`reservedAt`) store dates as `String`. `Loan.isOverdue()` and the services call `LocalDate.now()` directly.
- **Why it is wrong:** the columns have no date type, and the code parses the text again at each use. The business logic reads the machine clock.
- **Impact:** a test cannot pin "today", so a date-dependent test is not reproducible.
- **Fix:** use `LocalDate` fields with an ORMLite persister, and inject a `Clock`.
- **Test:** none. The fixed tests inject a fixed `Clock`.

5.12 Error #11: Hardcoded overdue fee rate

- **Location:** `service/LoanService.java: returnLoan(...)` computes `daysOverdue * 1.0`. No other method computes a fee.
- **Why it is wrong:** the fee policy (1.0 per day past due) is a magic number. A policy change needs an edit in the service, not in one policy class. The list endpoints also report 0.00 for an open overdue loan, so the fee stays hidden until the return.
- **Impact:** `GET /loans` and `GET /loans/overdue` show a 0.00 fee for an open overdue loan. The library only sees the debt after the member returns the book.
- **Fix:** add `LoanPolicy.FEE_PER_DAY` next to `DUE_DAYS`, and compute the accrued fee when the service lists open overdue loans.
- **Test:** none.

5.13 Error #12: returnLoan hides a missing or returned loan

- **Location:** `service/LoanService.java: returnLoan(...)` returns the loan when the loan is null or already returned.
- **Why it is wrong:** the method reports success for a missing loan and for an already-returned loan. The caller cannot separate the two cases.
- **Impact:** `POST /loans/9999/return` answers 200 with a `null` body. A second return of the same loan answers 200 with the loan, so the caller cannot tell success from a no-op.
- **Fix:** throw `NotFoundException` for a missing loan and map it to 404. Throw `IllegalStateException` for an already-returned loan and map it to 409.
- **Test:** none.

6. Grading rubric

The work has two stages. One student does Stage 1 first, then Stage 2. Part 1 grades Stage 1. Part 2 grades Stage 2. Each stage ends with a build and a test run. The total is 80 points. Each score reflects the complexity of the item, not the number of lines of code. Complexity runs from 1 (mechanical) to 5 (hardest). The change numbers refer to the change summary in Section 2.1.

6.1 Part 1: Data layer and policy (Stage 1, 38 points)

Stage 1 changes the model, the persistence layer, and the shared infrastructure. The services keep their current shape, but they must compile against the new types. `DueDateDuplicationTest` must pass. `TransactionBugTest` can still fail: Stage 2 fixes the transactions.

#	Change	Done when	Complexity	Points
1a	License header and javadoc	Every file that Stage 1 touches has the header. Every type, field, method, and parameter has javadoc.	1	1
2	Typed dates	The date fields are <code>LocalDate</code> . <code>LocalDatePersister</code> exists. <code>Loan.isOverdue()</code> is gone.	4	8
2b	Call sites	<code>MemberService</code> , <code>LoanService</code> , <code>ReservationService</code> , and <code>Database</code> compile against the <code>LocalDate</code> fields.	2	2
2c	Stage 1 test	<code>DueDateDuplicationTest</code> passes.	2	1
3	Loan policy	<code>LoanPolicy</code> holds <code>DUE_DAYS</code> , <code>FEE_PER_DAY</code> , and <code>dueDate(...)</code> . Checkout and fulfill use it.	1	1
4	Not found error	<code>NotFoundException</code> extends <code>RuntimeException</code> .	1	1
5	Base DAO	The four DAOs extend <code>BaseDao</code> . <code>BaseDao</code> has the CRUD methods and <code>transaction</code> .	4	8
6	Inventory guard	<code>BookService.borrow()</code> rejects a book with no copies.	2	3
10	Seed data	The seed creates three members, one reservation, and three loans. No <code>createLoan</code> helper.	3	5
13	Browser overdue state	<code>app.js</code> computes <code>isOverdue</code> . <code>l.overdue</code> is gone.	1	1
14	Logback level	<code>logback.xml</code> has the license comment and the <code>cl.ucn.disc</code> DEBUG logger.	1	1
16a	Class diagram	<code>docs/class-diagram.puml</code> shows <code>BaseDao</code> , <code>LocalDatePersister</code> , <code>LoanPolicy</code> , and <code>NotFoundException</code> . The date fields are <code>LocalDate</code> . <code>Loan.isOverdue()</code> is gone.	1	1

#	Change	Done when	Complexity	Points
AI	AI prompt log	<code>docs/prompts-stage-1.md</code> holds every prompt of Stage 1, in order. Each prompt gives the model name, the exact prompt text, and the files that changed.	1	5

Stage 1 removes Error #5. It also adds the guard of Error #6, the persister of Error #10, and the transaction helper of Errors #3 and #8.

AI prompt log (Stage 1)

Submit the prompts in `docs/prompts-stage-1.md`. Write the prompts in order. For each prompt, write the model name, the exact prompt text, and the files that changed. Copy the prompt text verbatim. A missing file loses 5 points. A log that does not match the code loses the same points.

6.2 Part 2: Services and integration (Stage 2, 42 points)

Stage 2 moves the lending logic into the services, adds the clock and the transactions, rewires the application, and updates the tests. All four tests must pass at the end of Stage 2.

#	Change	Done when	Complexity	Points
1b	License header and javadoc	Every file that Stage 2 touches has the header. Every type, field, method, and parameter has javadoc.	1	1
7	Member service cleanup	<code>MemberService</code> has one DAO. <code>checkout()</code> is gone.	1	2
8	Loan service	<code>LoanService</code> owns <code>checkout()</code> , receives a <code>Clock</code> , and uses transactions. <code>returnLoan()</code> rejects a missing loan and a returned loan.	5	12
9	Reservation service	<code>fulfill()</code> uses a transaction and <code>BookService.borrow()</code> . The <code>BookDao</code> field is gone.	4	8
11	App wiring	<code>App</code> uses <code>database.db</code> , the Santiago clock, the new wiring, the date mapper, and the exception handlers.	3	5
12	Controllers	The controllers call the services. No <code>BookDao</code> , no <code>MemberService</code> , no <code>Objects.requireNonNull</code> .	2	3
15	Tests	The two updated tests pass. <code>AccruedFeeTest</code> and <code>DatabaseSeedTest</code> exist and pass.	3	5
16b	Class diagram	<code>docs/class-diagram.puml</code> shows the new service fields and methods and the controller dependencies.	1	1

#	Change	Done when	Complexity	Points
AI	AI prompt log	<code>docs/prompts-stage-2.md</code> holds every prompt of Stage 2, in order. Each prompt gives the model name, the exact prompt text, and the files that changed.	1	5

Stage 2 removes Errors #1, #2, #3, #4, #6, #7, #8, #9, #11, and #12. It completes Error #10 with the clock injection.

AI prompt log (Stage 2)

Submit the prompts in `docs/prompts-stage-2.md`. Write the prompts in order. For each prompt, write the model name, the exact prompt text, and the files that changed. Copy the prompt text verbatim. A missing file loses 5 points. A log that does not match the code loses the same points.

Total: 80 points.

Appendix A: Prompt log format

The table below shows the format of `docs/prompts-stage-1.md` and `docs/prompts-stage-2.md`. Write one row for each prompt. Copy the prompt text verbatim. Do not write a summary of the prompt. The rows below are examples. Do not copy them. Write the exact model ID that you used.

Number	Model	Exact prompt text
1	deepseek-v4-flash	Read docs/refactor.md. Apply change 2. Create db/LocalDatePersister.java. Change the date fields of Loan and Reservation to LocalDate. Delete Loan.isOverdue(). Keep the project compilable.
2	claude-opus-4-6	Apply change 5 of docs/refactor.md. Create dao/BaseDao.java with findAll, findById, create, update, delete, and transaction. Make the four DAOs extend it.
3	gpt-5.2	Why does TransactionBugTest fail on the original code? Explain the partial write in MemberService.checkout.
4	claude-opus-4-6	Apply change 8 of docs/refactor.md. Move checkout() into LoanService. Inject a Clock. Run the two writes in one transaction.
5	gpt-5.2	TransactionBugTest still fails with SQLException: cannot commit transaction. Find the cause and fix it.
6	deepseek-v4-flash	Add the three exception handlers of change 11 to App.java. Map NotFoundException to 404, IllegalArgumentException to 400, and IllegalStateException to 409.
7	gemini-3-pro	Write AccruedFeeTest. Use an in-memory database and a fixed clock. Assert 24.0 for the overdue loan and 0.0 for the active loan.